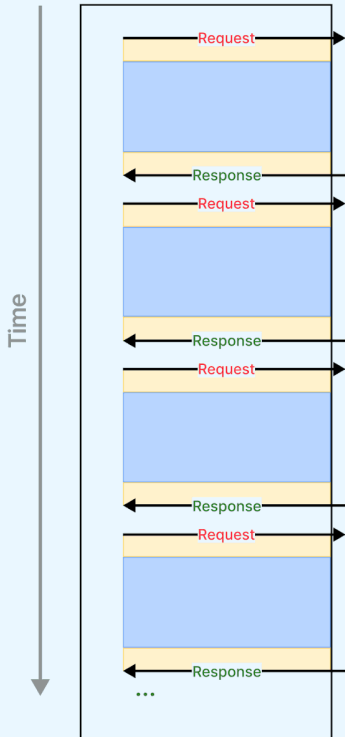
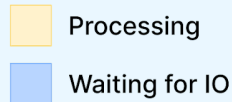
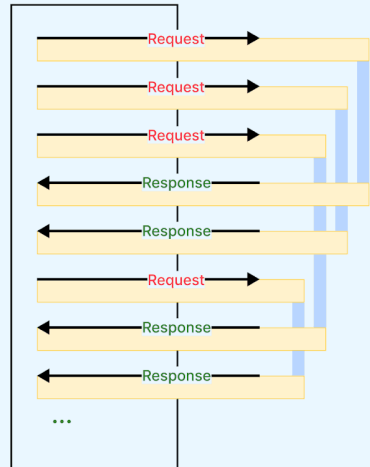


Async Python for AI

Synchronous Program



Asynchronous Program



LAMBDATEST



AI Coach John

Follow

PROITBRIDGE
Dinner For Better Future

Repost

🕒 THE PROBLEM

An LLM call is 99% waiting

When you call an LLM API, your program sends a request and then sits idle until the response comes back. That waiting is not computation it's your process blocked on the network.

WHAT ONE LLM CALL ACTUALLY LOOKS LIKE | | send

WAITING

parse 2ms

~4,000ms of doing absolutely nothing

3ms CPU usage: |

|

|

|

busy

idle, blocked on network

busy

100 calls, one after another:

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

... x 100

└─ 400 seconds of your program doing nothing at all ─┘

The core insight: this is an **I/O-bound** workload, not a CPU-bound one. Your bottleneck isn't computation it's waiting on someone else's server. Async exists precisely for this shape of problem.



AI Coach John

Follow



↻ Repost

Async only helps one of these

✓ I/O-BOUND (ASYNC HELPS)

- LLM API calls
- vector DB queries
- HTTP requests
- database reads
- file I/O
- waiting on other services

the CPU is IDLE while waiting
→ let it start other work

✗ CPU-BOUND (ASYNC DOESN'T HELP)

- matrix multiplication
- local model inference
- image processing
- computing embeddings
 locally on CPU
- parsing huge JSON

the CPU is BUSY the whole time
→ nothing to overlap with

🔗 THE RULE

Is your program WAITING on something external? ► asyncio

Is your program CALCULATING something? ► multiprocessing

Why this matters for AI specifically: a RAG pipeline is mostly I/O embed (API), retrieve (DB), rerank (API or local), generate (API). Almost every stage is a wait. That's why async has an outsized payoff here.



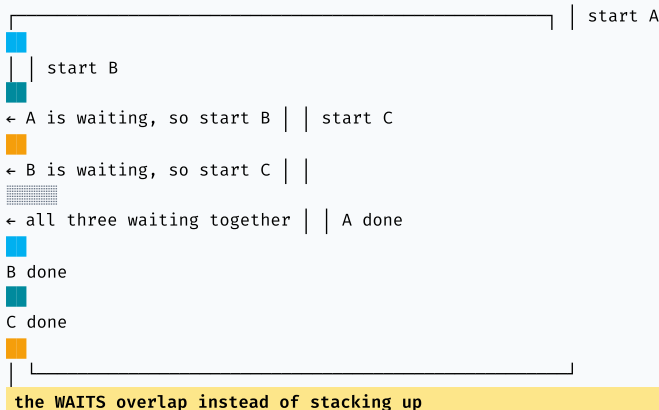
One thread, switching whenever something waits

Async doesn't use more threads or more CPUs. It uses **one** thread very efficiently when a task hits a wait, it hands control back so another task can start.

SYNCHRONOUS (one thread, blocking)



ASYNC (one thread, cooperative)



The one-sentence version: async makes the *waiting* happen in parallel, not the *computing*.



AI Coach John

Follow



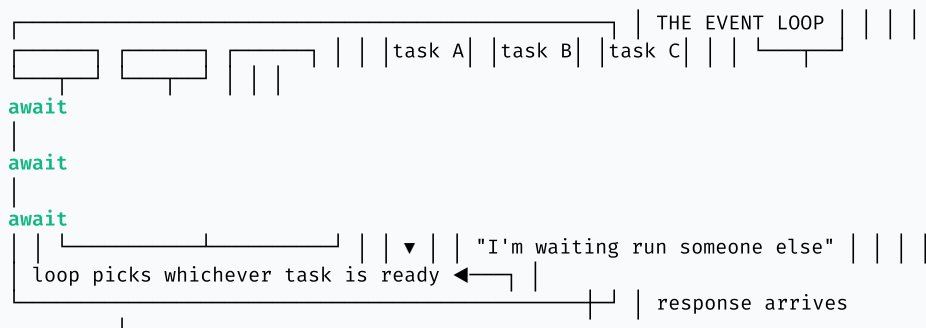
🔄 Repost

async, await, and the event loop

```
import asyncio

# async def → this function can pause and resume
async def fetch_answer(question: str) → str:
    # await → "this will take a while; let other tasks run"
    response = await llm.ainvoke(question)
    return response.content

# asyncio.run → starts the event loop, the scheduler that
#             decides which paused task resumes next
result = asyncio.run(fetch_answer("What is RAG?"))
```



The critical rule: `await` only yields control if the thing you're awaiting is *actually async*. Awaiting a blocking library gives you no benefit at all this is the single most common async mistake.



Side by side, 100 LLM calls

⌚ SYNCHRONOUS

```
import time

def process_all(questions):
    results = []
    for q in questions:
        # blocks ~4s each
        res = llm.invoke(q)
        results.append(res.content)
    return results

start = time.perf_counter()
answers = process_all(q_100)
print(f"{time.perf_counter() - start:.1f}s")
# ~400.0s
```

⚡ ASYNCHRONOUS

```
import asyncio

async def process_all(questions):
    # create, don't await yet
    tasks = [llm.ainvoke(q)
              for q in questions]
    # await ALL together
    res = await asyncio.gather(*tasks)
    return [r.content for r in res]

start = time.perf_counter()
answers = asyncio.run(process_all(q_100))
print(f"{time.perf_counter() - start:.1f}s")
# ~6.2s
```

THE TWO-LINE CHANGE THAT DOES IT `tasks = [llm.ainvoke(q) for q in questions]`

← build coroutines

`await asyncio.gather(*tasks)`

← run them together

▲ **note:** creating a coroutine does NOT start it. `asyncio.gather` is what actually launches them.



Four ways to run things concurrently

```
# — gather: run all, get results in ORDER —
results = await asyncio.gather(task_a(), task_b(), task_c())
# results[0] is always task_a's result

# — gather with error tolerance —
results = await asyncio.gather(*tasks, return_exceptions=True)
for r in results:
    if isinstance(r, Exception):
        log.error("task_failed", error=str(r)) # one failure ≠ total failure

# — as_completed: process results AS THEY ARRIVE —
for coro in asyncio.as_completed(tasks):
    result = await coro
    yield result # stream to the user immediately

# — TaskGroup (3.11+): structured, auto-cancels siblings on error —
async with asyncio.TaskGroup() as tg:
    t1 = tg.create_task(retrieve(query))
    t2 = tg.create_task(fetch_user_context(user_id))
# both guaranteed complete (or the whole group failed) here
```

? WHICH ONE DO YOU WANT?

need results in order, all must finish	▶ gather
one failure shouldn't kill the batch	▶ gather(return_exceptions=True)
want to stream results as they land	▶ as_completed
want clean cancellation on failure	▶ TaskGroup (3.11+)



Unlimited concurrency will get you rate-limited

❌ NO LIMIT

1000 requests fired
all at once

- 429 Too Many
- connection errors
- memory spike

✅ SEMAPHORE (LIMIT = 10)

 10 running
990 queued

- steady, no 429s
- predictable memory
- still ~100x faster

```
async def process_with_limit(questions: list[str], max_concurrent: int = 10):  
    semaphore = asyncio.Semaphore(max_concurrent)  
  
    async def bounded(q: str):  
        async with semaphore: # wait for a free slot  
            return await llm.ainvoke(q)  
  
    return await asyncio.gather(*[bounded(q) for q in questions])
```

How to pick the number: start conservative (5-10), watch for 429s, and raise it until you approach your provider's rate limit. More concurrency past that point gives you errors, not speed.



Async retries that don't block everything else

```
async def call_with_retry(question: str, max_retries: int = 4):
    for attempt in range(max_retries):
        try:
            return await llm.ainvoke(question)

        except RateLimitError:
            wait = 2 ** attempt          # 1s, 2s, 4s, 8s
            log.warning("rate_limited", attempt=attempt, wait_s=wait)
            await asyncio.sleep(wait)    # ← async sleep, non-blocking

        except (TimeoutError, ProviderError) as e:
            if attempt == max_retries - 1:
                raise
            await asyncio.sleep(2 ** attempt)

    raise RuntimeError("max retries exceeded")
```

⚠ THE MISTAKE THAT KILLS ASYNC

`time.sleep(2)` ✗ blocks the ENTIRE event loop all 99 other tasks freeze too
`await asyncio.sleep(2)` ✓ only this task pauses everything else keeps running



One synchronous line ruins the whole thing

YOUR "ASYNC" PIPELINE

```
async def
process
(q): docs = vector_store.search(q)
← SYNC, blocks everything ⚠️
answer =
await
llm.ainvoke(q)
← async ✓
requests.post(webhook, ...)
← SYNC, blocks everything ⚠️
return
answer
```



← still effectively | | ▲▲ sequential! | |

blocking
blocking

you added async keywords and gained almost nothing

```
# 1. Use the async version of the library (best)
docs = await vector_store.asearch(q)
async with httpx.AsyncClient() as client:
    await client.post(webhook, json=payload)

# 2. No async version available? Offload to a thread
docs = await asyncio.to_thread(vector_store.search, q)

# 3. CPU-heavy work? Use a process pool, not a thread
loop = asyncio.get_running_loop()
result = await loop.run_in_executor(process_pool, heavy_computation, data)
```

✖ requests

✖ time.sleep

✖ open() (large files)

✖ sync SDKs



AI Coach John

Follow



🔄 Repost

Where the wins actually are

```

async def rag_pipeline(query: str) → str:
    # 1. Independent work → run CONCURRENTLY
    query_vec, user_ctx = await asyncio.gather(
        embedder.aencode(query), fetch_user_context(user_id)
    )

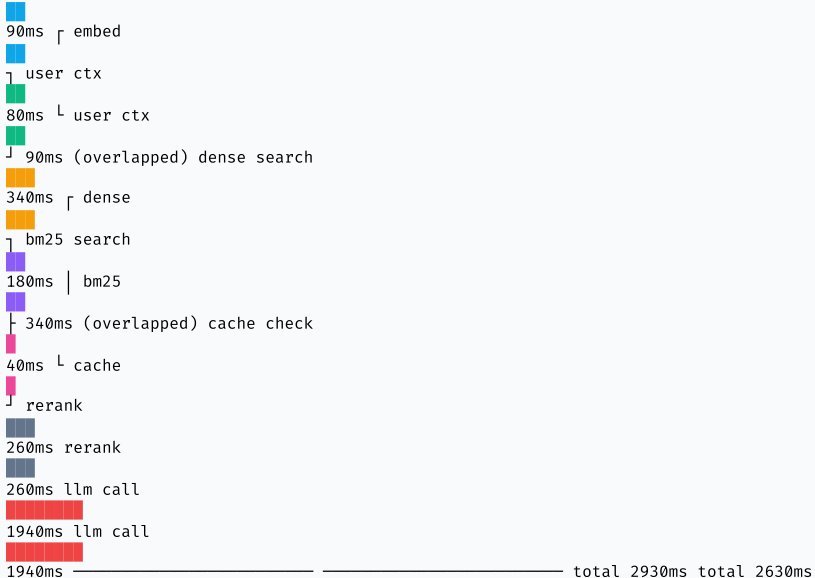
    # 2. Search multiple sources at once
    dense_docs, sparse_docs, cached = await asyncio.gather(
        vector_store.asearch(query_vec, k=20),
        bm25_index.asearch(query, k=20),
        semantic_cache.aget(query),
        return_exceptions=True, # cache miss shouldn't kill request
    )

    # 3. Rerank chunks in parallel batches
    scores = await asyncio.gather(*[
        reranker.ascore(query, batch) for batch in chunk_batches
    ])

    # 4. Sequential by necessity needs the results above
    return await llm.ainvoke(build_prompt(query, top_docs))

```

SEQUENTIAL PIPELINE CONCURRENT WHERE POSSIBLE embed



Tokens to the user as they're generated

— Async generator: yields tokens as they arrive —

```
async def stream_answer(query: str):
    docs = await retrieve(query)
    prompt = build_prompt(query, docs)

    async for chunk in llm.astream(prompt):
        yield chunk.content
```

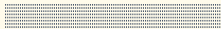
— Wired into FastAPI —

```
from fastapi.responses import StreamingResponse
```

```
@app.post("/query")
```

```
async def query_endpoint(request: QueryRequest):
    return StreamingResponse(
        stream_answer(request.query),
        media_type="text/event-stream",
    )
```

⚙️ WITHOUT STREAMING



2.8s of nothing
then the full answer

user stares at a spinner

⚡ WITH STREAMING



▲ first token 400ms
then text flows

user is already reading

same total time completely different experience

Why this needs async: streaming means holding a connection open while tokens trickle in. Under async, one process can hold hundreds of open streams.



The classic async AI workload

```

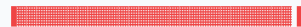
async def process_corpus(docs: list[str], concurrency: int = 20):
    semaphore = asyncio.Semaphore(concurrency)
    completed = 0

    async def process_one(doc: str):
        nonlocal completed
        async with semaphore:
            try:
                result = await call_with_retry(doc)
            except Exception as e:
                log.error("doc_failed", error=str(e))
                return None
            finally:
                completed += 1
                if completed % 100 == 0:
                    log.info("progress", done=completed, total=len(docs))
            return result

    results = await asyncio.gather(*[process_one(d) for d in docs])
    return [r for r in results if r is not None]

```

10,000 DOCUMENTS, 4s PER CALL sequential



~11 hours

async, limit 20



~33 minutes

async, limit 50



~13 minutes

▲ past this point you hit provider rate limits, not speed gains

Three things this code gets right: a concurrency cap so you don't self-DoS, per-item error isolation so one bad document doesn't lose 9,999 good results, and **progress logging** so a 30-minute job isn't a black box.



AI Coach John

Follow



🔄 Repost

One hung request shouldn't stall a batch

```
# — Per-call timeout —
try:
    result = await asyncio.wait_for(llm.ainvoke(query), timeout=30.0)
except asyncio.TimeoutError:
    log.warning("llm_timeout", query_id=qid)
    result = fallback_response()

# — Whole-operation timeout (3.11+, cleaner) —
async with asyncio.timeout(60):
    docs = await retrieve(query)
    answer = await generate(query, docs)
    # if the WHOLE block exceeds 60s, it cancels

# — Race two providers, take whichever answers first —
done, pending = await asyncio.wait(
    [asyncio.create_task(primary.ainvoke(q)),
     asyncio.create_task(fallback.ainvoke(q))],
    return_when=asyncio.FIRST_COMPLETED,
)
for task in pending: task.cancel() # don't pay for the loser
result = done.pop().result()
```

✗ NO TIMEOUT

99 done in 8s
1 hangs forever

**gather() never
returns ⚠**

✓ WITH TIMEOUT

99 done in 8s
1 cancelled at 30s

batch completes
with 99 results ✓



Async does not speed up computation

API CALL (I/O-BOUND)



▲ waiting on network
async overlaps this ✓

LOCAL INFERENCE (CPU/GPU-BOUND)



▲ GPU busy whole time
nothing to overlap ✗

What to do instead for local models:

```
# ✗ WRONG: wrapping local inference in async gains nothing
async def bad(text):
    return local_model.generate(text) # still blocks the loop

# ✓ Offload so it at least doesn't block other async work
async def better(text):
    return await asyncio.to_thread(local_model.generate, text)

# ✓✓ BEST: batch at the model level, where the real speedup is
outputs = local_model.generate(list_of_texts, batch_size=16)
```

The key distinction: for local inference, **batching** is your concurrency mechanism, not `asyncio`. A serving layer like `vLLM` handles continuous batching internally `asyncio` is only useful for the HTTP layer in front of it.



Where this pays off in a real API

```
from fastapi import FastAPI
app = FastAPI()

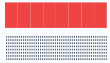
# X Sync endpoint: FastAPI runs it in a thread from a limited pool.
# Under load, the pool exhausts and requests queue.
@app.post("/query-sync")
def query_sync(req: QueryRequest):
    return {"answer": llm.invoke(req.query).content}

# ✓ Async endpoint: one process handles many concurrent requests
@app.post("/query")
async def query(req: QueryRequest):
    return {"answer": (await llm.ainvoke(req.query)).content}

# ✓ Reuse one client across requests don't create per-request
client = httpx.AsyncClient(timeout=30.0)

@app.on_event("shutdown")
async def cleanup():
    await client.aclose()
```

CONCURRENT REQUESTS ON ONE PROCESS sync endpoint



~thread-pool limited async endpoint



hundreds of in-flight requests same hardware • same code logic •
different keyword

A subtle trap: an `async def` endpoint containing a blocking call is **worse** than a plain `def` endpoint FastAPI would have safely put the `def` one in a thread, but it trusts your `async def` not to block, and one blocking call stalls every concurrent request.



Async bugs look different

```
# 1. Turn on debug mode warns about slow/blocking callbacks
asyncio.run(main(), debug=True)
# "Executing <Task ...> took 2.431 seconds" ← found your blocking call

# 2. Name your tasks so tracebacks are readable
task = asyncio.create_task(process(doc), name=f"process_doc_{doc.id}")

# 3. See what's currently running
for t in asyncio.all_tasks():
    print(t.get_name(), t.get_coro())

# 4. NEVER swallow exceptions silently in a fire-and-forget task
asyncio.create_task(bg_job()) # X errors vanish

task = asyncio.create_task(bg_job()) # ✓
task.add_done_callback(
    lambda t: log.error("bg_failed", error=str(t.exception()))
    if t.exception() else None
)
```

SYMPTOM	USUALLY MEANS
"async but not faster"	a blocking call in the path
"429 rate limit errors"	no semaphore / concurrency cap
"gather() never returns"	no timeout on a hung request
"RuntimeError: no event loop"	calling async code from sync context
"coroutine never awaited"	forgot await / built but never gathered
"silent background failures"	fire-and-forget task, no done callback



Everything from this carousel, assembled

```
class AsyncLLMClient:
    def __init__(self, llm, max_concurrent=10, timeout=30.0, max_retries=3):
        self.llm = llm
        self.sem = asyncio.Semaphore(max_concurrent) # 1. concurrency cap
        self.timeout = timeout
        self.max_retries = max_retries

    async def call(self, prompt: str):
        async with self.sem: # 1
            for attempt in range(self.max_retries):
                try:
                    return await asyncio.wait_for( # 2. timeout
                        self.llm.ainvoke(prompt), self.timeout
                    )
                except RateLimitError:
                    await asyncio.sleep(2 ** attempt) # 3. async backoff
                except asyncio.TimeoutError:
                    log.warning("timeout", attempt=attempt)
                    raise RuntimeError("all retries exhausted")

    async def call_many(self, prompts: list[str]):
        results = await asyncio.gather(
            *[self.call(p) for p in prompts],
            return_exceptions=True, # 4. error isolation
        )
        return [r for r in results if not isinstance(r, Exception)]
```

≡ THE FIVE THINGS EVERY ASYNC AI CLIENT NEEDS

1. **concurrency cap** ► Semaphore
2. **timeout per call** ► `asyncio.wait_for`
3. **retry with backoff** ► `await asyncio.sleep(2 ** n)`
4. **error isolation** ► `gather(return_exceptions=True)`
5. **no blocking calls** ► `async libs` or `asyncio.to_thread`



⚠ COMMON MISTAKES

What goes wrong with async in AI code

- ⚠ Using `time.sleep` instead of `await asyncio.sleep`
- ⚠ Calling a sync library inside an async function
- ⚠ No concurrency limit instant 429s and memory spikes
- ⚠ No timeout one hung call stalls the whole gather
- ⚠ Creating an HTTP client per request instead of reusing one
- ⚠ Forgetting `return_exceptions=True` one failure loses the batch
- ⚠ Fire-and-forget tasks with no error callback
- ⚠ Expecting async to speed up local model inference
- ⚠ `async def` endpoint with a blocking call (worse than plain `def`)

💡 THE ONE-LINE TEST

"Am I WAITING on someone else?" ► **async will help** ✓ "Am I CALCULATING something?" ► **async won't help** ✗ If you can't answer this for a given line of code, you don't yet know whether to await it.



AI Coach John

Follow



↻ Repost

Make the waiting overlap

An LLM call is 99% waiting on a network. Async doesn't make any single call faster it lets a hundred calls wait *at the same time* instead of one after another. Add a concurrency cap so you don't get rate-limited, timeouts so one hung call can't stall a batch, retries with async backoff, and error isolation so one failure doesn't cost you the other ninety-nine.

Semaphore

▶ don't get 429'd

`asyncio.wait_for`

▶ don't hang

`await asyncio.sleep`

▶ don't block

`return_exceptions=True`

▶ don't lose a batch

`async libs / to_thread`

▶ don't fake it

100 calls · 400 seconds ▶ 6 seconds

Save this as your async AI reference.

Follow for more practical AI engineering breakdowns.



AI Coach John

[Follow](#)



[↻ Repost](#)